Practical Work (TP 1)

Introduction to Information Retrieval & Search Engine Architecture

Course	Information Web Retrieval and Analytics
Level	Engineering
Topic	Basics of IR, differences with databases, and search engine architecture

Name:	HEAM Visal	**Student ID:**	e20221349
Group:	I5-AMS-DAS	**Date:**	14/10/2026

Instructions

- Answer all **5 questions** in this document, in your own words.
- Submit **one PDF** by email (my email: dorn.dawin@gmail.com) and named TP1_StudentID_Name.pdf by **[14/05/2026 : 11:59PM]**.
- Work individually. If you use an AI tool, declare it at the end and copy or screenshot your prompt.

Question	Topic	Marks
1	Information Retrieval vs. Relational Databases	20
2	Text Transformation Pipeline	20
3	Search Engine Architecture & Processes	20
4	Inverted Index	20
5	Basic Scoring Model	20
	Total	**100**

Question 1 · Information Retrieval vs. Relational Databases (20 pts)

Compare Information Retrieval (IR) systems with traditional Relational Database Management Systems (RDBMS) based on the following criteria. Give an example for each criterion.

Criterion	Relational database (RDBMS)	Information retrieval (IR) system
1. Data type / structure	Store structure data in table with predefined row and columns. Example: Bank records containing account number, name, and balance.	Store and searches mainly unstructured or semi structure data, such as text documents Example: Web pages, PDFs, and news articles.
2. Querying & matching strategy	Use SQL queries and exact matching. A record either satisfies the conditional or does not. Example: SELECT balance FROM accounts WHERE account_no = 10423;	Use keyword queries and partial matching based on relevance. Example: Searching "cheap flight Phnom Penh Paris" returns related documents ranked by relevance.
3. Output / result format	Return a set of matching rows (records) in a table. Example: A result containing the account number and balance.	Return a ranked list of documents, with the most relevant results first. most relevant results first. Example: Search results listing relevant flight-booking websites.

Question 2 · Text Transformation Pipeline (20 pts)

During the **Text Transformation** phase of the indexing process, text undergoes several preprocessing steps. Trace the following raw HTML string through each step of the pipeline:

Raw input:<h1>Library News</h1> The students are searching the libraries for useful books.

Use these rules:

- **Tokenizer:** convert to lower case and remove punctuation.
- **Stop list:** a, an, and, are, as, at, be, by, for, from, in, is, it, of, on, or, the, to, was, were, with.
- **Stemmer:** the Porter stemmer. You can check stems with an online Porter stemmer or with Python: from nltk.stem import PorterStemmer.

List the **explicit list of terms** output at each stage, and explain in one or two sentences what the step did.

Stage	Output (list of terms)	Explanation
1. Parser & Tokenizer identify structural markup, remove tags, produce tokens	[library, news, the, students, are, searching, the, libraries, for, useful, books]	The parser identifies and removes HTML tag (h1, p). The Tokenizer converts the text into lowercase, remove the punctuation, and splits it into individual words.
2. Stop word removal remove common functional words	[library, news, students, searching, libraries, useful, books]	Remove common words such as “the”, “are” and “for” based on the given stop list. This keeps the important terms.
3. Stemming reduce words to their root or stem forms	[librari, news, student, search, librari, use, book]	Use the porter stemmer to reduces words to their stem forms. Ex: libraries to librari

Question 3 · Search Engine Architecture & Processes (20 pts)

A modern search engine architecture is divided into two primary sub-processes: the **Indexing Process** and the **Query Process**.

3.1 Name and explain the core responsibilities of the **three components in the Indexing Process**. (7 pts)

Component	Core responsibilities
Text Acquisition	Identifies, collects, and stores documents from sources such as web pages, emails, and files for indexing.
Text Transformation	Processes and transforms documents into index terms or features through parsing, tokenization, stop-word removal, and stemming.
Index Creation	Creates data structures, such as inverted indexes, from the extracted term to support fast the efficient searching.

3.2 Name and explain the core responsibilities of the **three components in the Query Process**.

(7 pts)

Component	Core responsibilities
User Interaction	Allows users to enter and refine search queries and displays the search results.
Ranking	Uses the query and indexes to calculate relevance scores and generate a ranked list of documents, with the most relevant result first.
Evaluation	Monitors and measures the effectiveness and efficiency of search engine, using query logs, ranking analysis, and performance analysis.

3.3 Discuss the fundamental trade-off search engine architects face between **Effectiveness** and **Efficiency**. (6 pts)

Effectiveness refers to the quality and relevance of search results, measured by precision and recall.

Efficiency refers to how quickly the search engine process queries, measured by response time (latency) and throughput (queries per second)

The trade-off is improving effectiveness with more complex ranking algorithms may require more processing time and resources, reducing efficiency. On the other hand, using simpler and faster methods may improve efficiency but reduce result quality. Therefore, search engines must balance effectiveness and efficiency to provide relevant results quickly.

Question 4 · Inverted Index (20 pts)

Consider the following collection of four documents:

Doc	Text
D1	web search engines rank web pages
D2	search engines index documents
D3	users search the web for documents
D4	engines crawl pages and index pages

4.1 Explain what **term inversion** means in index creation: moving from **Document → Terms** to **Term → Documents**. (5 pts)

Term inversion is the process of converting a Document → Terms representation into a Term → Documents representation.

Instead of storing which terms appear in each document, an inverted index stores each term with a list of documents containing it (postings list). This allows the search engine to quickly find relevant documents without scanning the entire collection.

4.2 Build the **inverted index** for this collection. Remove the stop words from the Question 2 stop list, but do **not** stem. List the terms in alphabetical order and, for each term, its postings with the term frequency, e.g. web → D1 (2), D3 (1). (10 pts)

Term	Postings: document (term frequency)
crawl	D4(1)
documents	D2(1), D3(1)
engines	D1(1), D2(1), D4(1)
index	D2(1), D4(1)
pages	D1(1), D4(2)
rank	D1(1)
search	D1(1), D2(1), D3(1)
users	D3(1)
web	D1(2), D3(1)

4.3 Using **only your inverted index** (not the documents), find the documents that match each Boolean query, and explain how you combined the postings lists. (5 pts)

Query	Matching documents	How you combined the postings
search AND engines	D1, D2	Intersection of search is D1, D2, D3 and engines is D1, D2, D4 → D1, D2
web AND documents	D3	Intersection of web is D1, D3 and documents is D2, D3 → D3
index OR crawl	D2, D4	Union of index is D2, D4 and crawl is D4 → D2, D4

Question 5 · Basic Scoring Model (20 pts)

In a simple vector/weighted IR model, the relevance score between a query q and a document d for shared terms i is calculated using:

$$\text{Score}(q, d) = \Sigma_i \, q_i \cdot d_i$$

where q_i is the query term weight and d_i is the document term weight.

5.1 Given a query vector **q = [1, 2]** and two document vectors **d_1 = [3, 0]** and **d_2 = [1, 2]**, calculate Score(q, d_1) and Score(q, d_2). Which document will be ranked higher? Explain why. (8 pts)

	Calculation	Score
Score(q, d_1)	(1 * 3) + (2 * 0) = 3 + 0	3
Score(q, d_2)	(1 * 1) + (2 * 2) = 1 + 4	5

Document d_2 ranks higher because its relevance score is 5, compared to d_1's score of 3. Document d_2 matches both query terms, including the second term, which has a higher query weight.

5.2 The table gives term weights for three documents over the vocabulary **[search, engine, web, index]**. The query is **q = [1, 1, 0, 2]**. Calculate the score of each document and give the final ranking. (12 pts)

	search	engine	web	index
q	1	1	0	2
d_1	3	0	2	0
d_2	1	2	0	1
d_3	0	1	1	3

Document	Calculation	Score	Rank
d_1	(1 * 3) + (1 * 0) + (0 * 2) + (2 * 0) = 3 + 0	3	3
d_2	(1*1) + (1*2) + (0*0) + (2*1) = 1 + 2 + 0 + 2	5	2
d_3	(1*0) + (1*1) + (0*1) + (2*3) = 0 + 1 + 0 + 6	7	1

Index Ranking :

$$d_3\ (7) > d_2\ (5) > d_1\ (3)$$

Declaration of AI tool use

I used the following AI tools for this TP (write “none” if you used none), and for which questions:

I use AI tool to help me answer and explain the question on questions 1, 2, and 3.

Prompt: review the lesson materials and sources, then provide the answers I should enter into the table, with explanations where needed.